

CNN Variants

Luca Benini

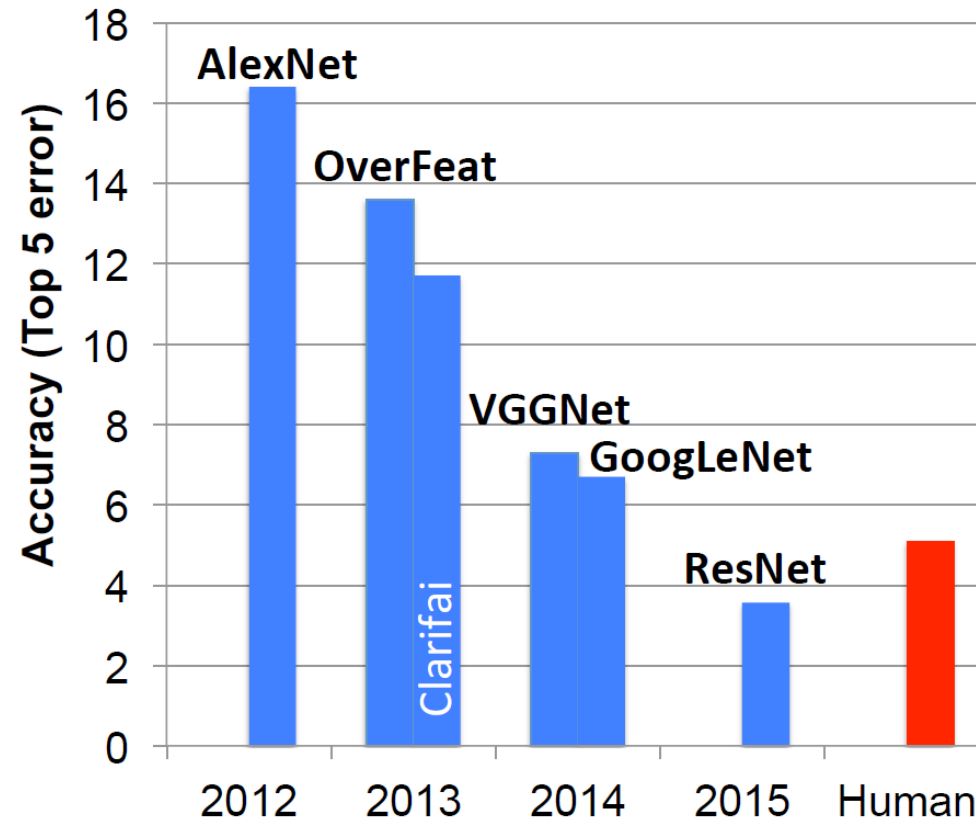
<lbenini@iis.ee.ethz.ch,luca.benini@unibo.it>

Popular DNNs

- **LeNet (1998)**
- **AlexNet (2012)**
- **OverFeat (2013)**
- **VGGNet (2014)**
- **GoogLeNet (2014)**
- **ResNet (2015)**

Now the challenge is considered solved as the error is more related to the error in the labels rather than in the networks

ImageNet: Large Scale Visual Recognition Challenge (ILSVRC)



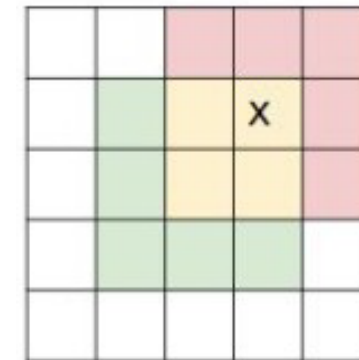
VGG-16

CONV Layers: **13**
Fully Connected Layers: **3**
Weights: **138M**
MACs: **15.5G**
Top5 error (ImageNet): **7.4**

Also, 19 layer version

Reduce # of weights

stack 2
3x3 conv



for a 5x5
receptive field

[figure credit
A. Karpathy]

More Layers → Deeper!

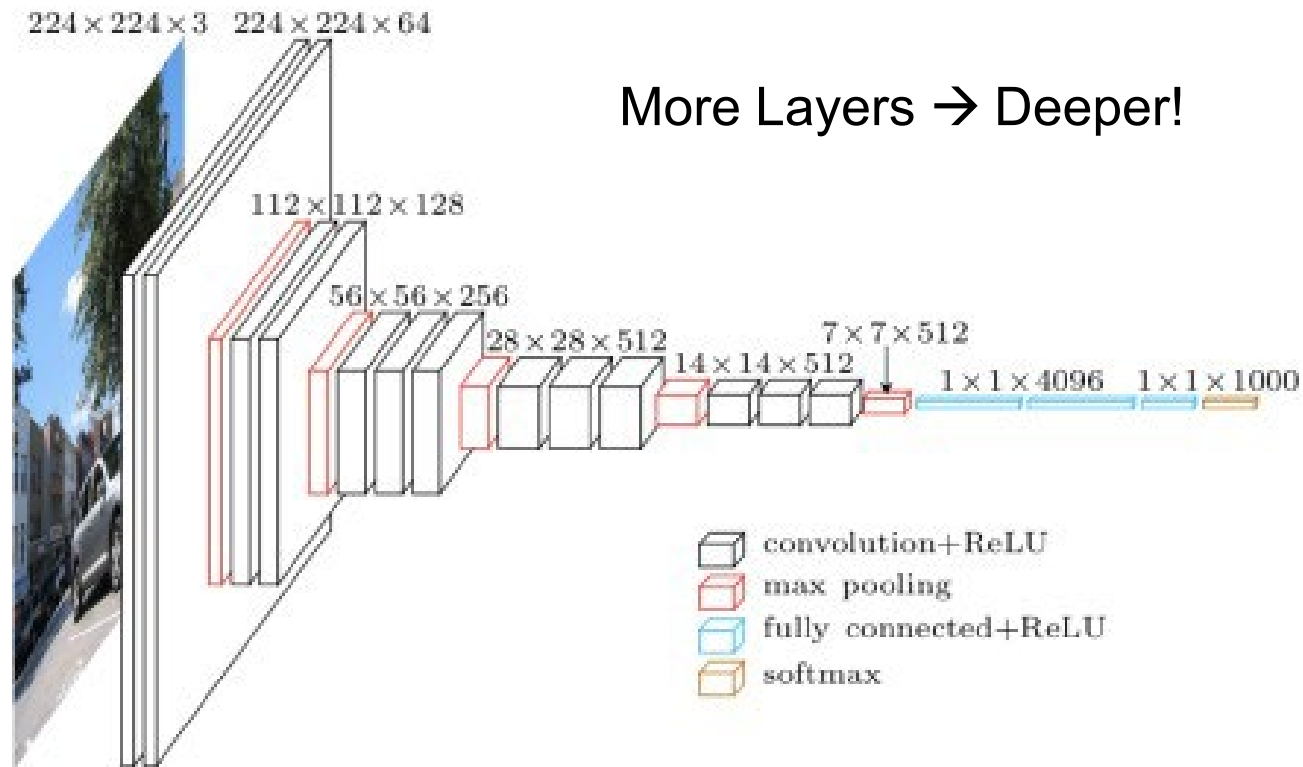


Image Source: <http://www.cs.toronto.edu/~frossard/post/vgg16/>

GoogLeNet (v1)

CONV Layers: **21 (depth), 57 (total)** Fully

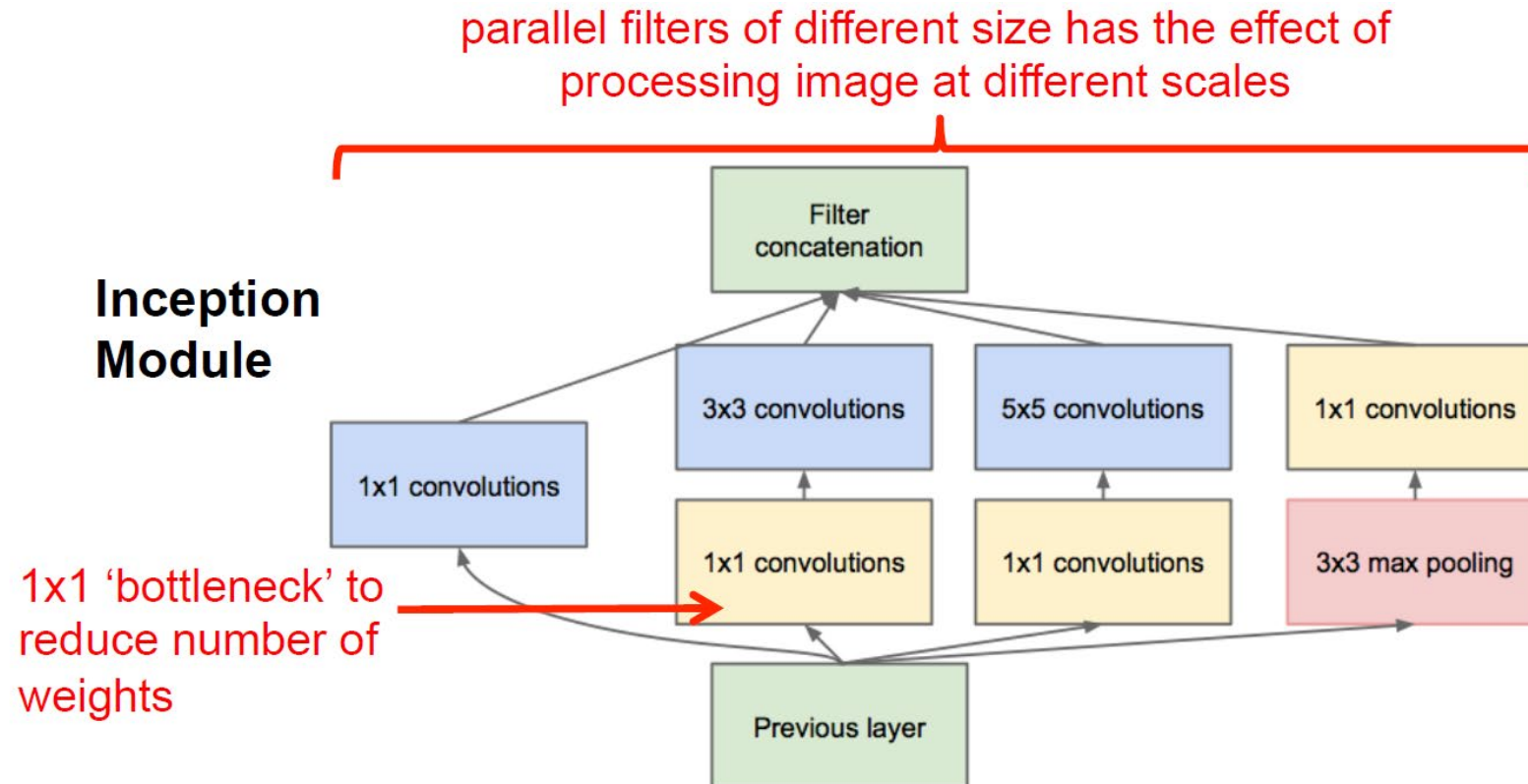
Connected Layers: **1**

Weights: **7.0M**

MACs: **1.43G**

Top5 error (Imagenet): **6.7**

Also, v2, v3 and v4
ILSVRC14 Winner



GoogLeNet (v1)

CONV Layers: 21 (depth), 57 (total)

Fully Connected Layers: 1

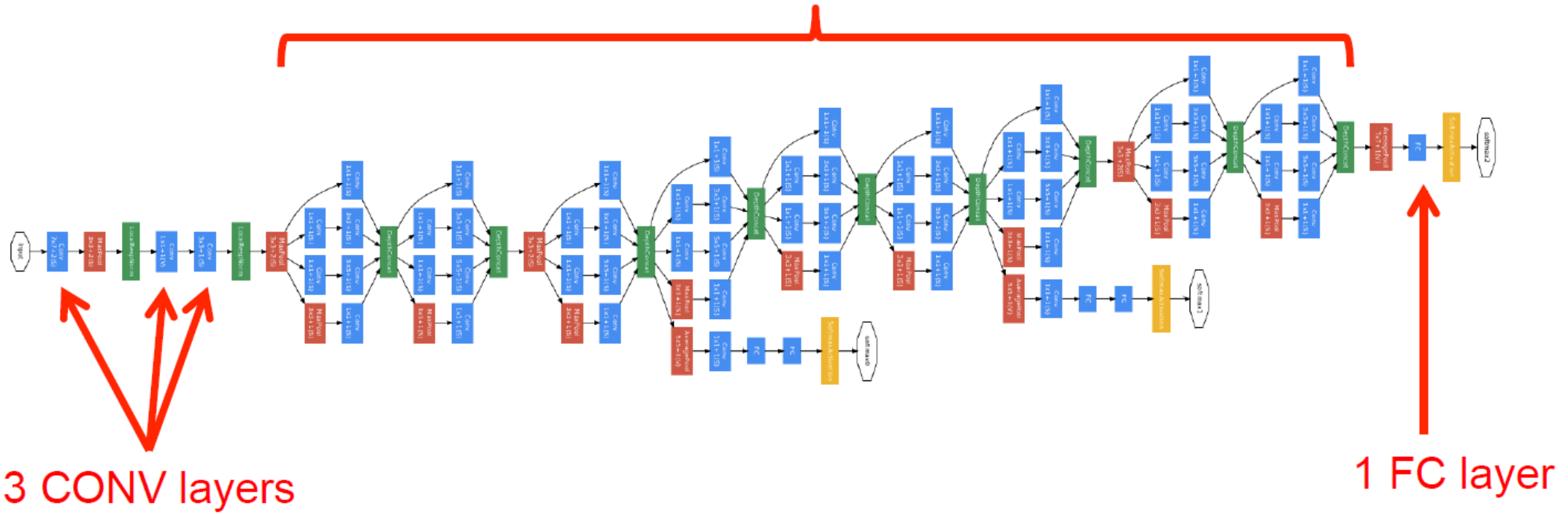
Weights: 7.0M

MACs: 1.43G

Also, v2, v3 and v4

ILSVRC14 Winner

9 Inception Layers



ResNet-50

CONV Layers: **49**

Fully Connected Layers: **1**

Weights: **25.5M**

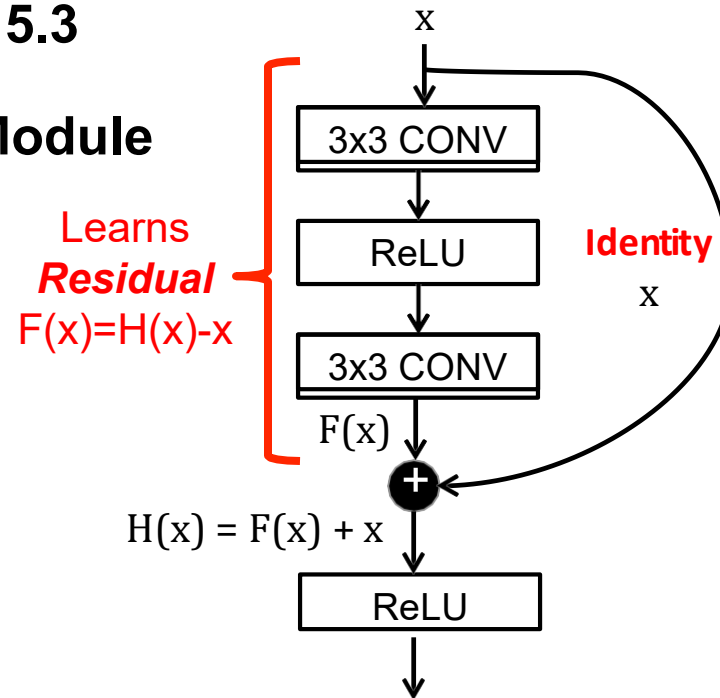
MACs: **3.9G**

To5 error (ImageNet): **5.3**

Also, 34, **152** and 1202 layer versions

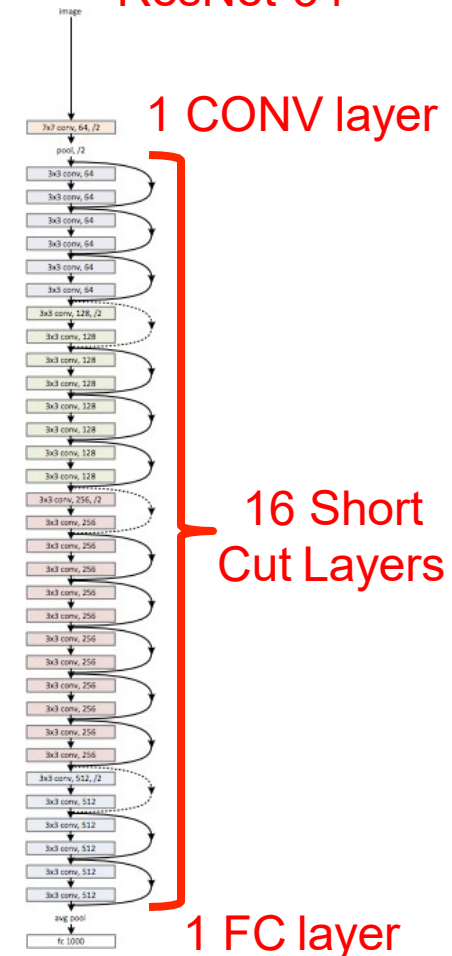
ILSVRC15 Winner

Short Cut Module



Helps address the vanishing gradient challenge for training very deep networks

ResNet-34



Revolution of Depth

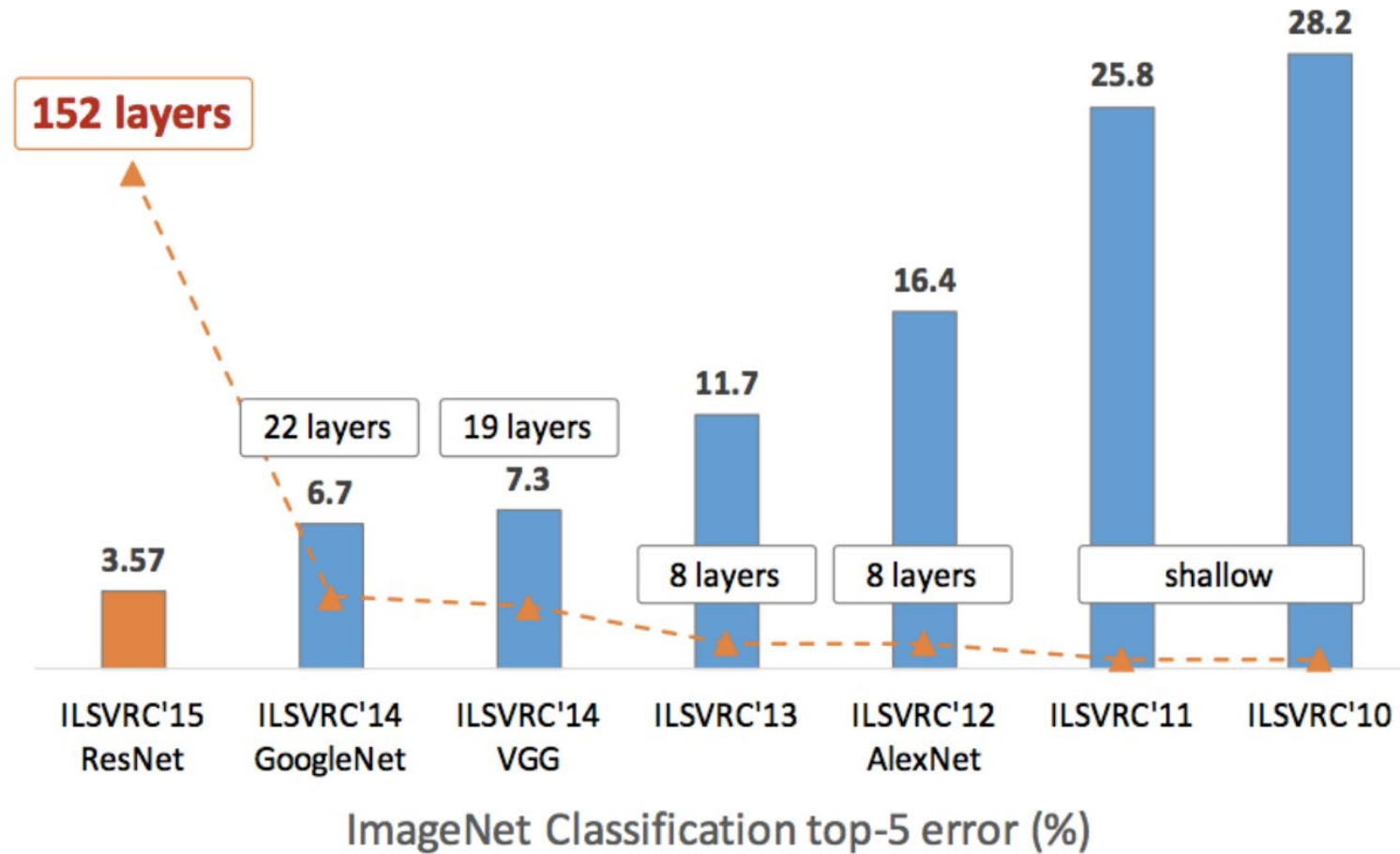


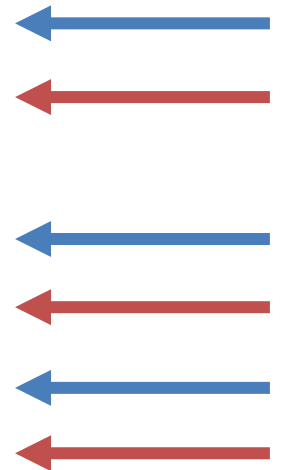
Image Source: http://icml.cc/2016/tutorials/icml2016_tutorial_deep_residual_networks_kaiminghe.pdf

Summary of Popular DNNs

- **AlexNet**
 - First CNN Winner of ILSVRC
 - Uses LRN (deprecated after this)
- **VGG-16**
 - Goes Deeper (16+ layers)
 - Uses only 3x3 filters (stack for larger filters)
- **GoogLeNet (v1)**
 - Reduces weights with Inception and only one FC layer
 - Inception: 1x1 and DAG (parallel connections)
 - Batch Normalization
- **ResNet**
 - Goes Deeper (24+ layers)
 - Shortcut connections

Summary of Popular DNNs

Metrics	LeNet-5	AlexNet	VGG-16	GoogLeNet (v1)	ResNet-50
Top-5 error	n/a	16.4	7.4	6.7	5.3
Input Size	28x28	227x227	224x224	224x224	224x224
# of CONV Layers	2	5	16	21 (depth)	49
Filter Sizes	5	3, 5, 11	3	1, 3, 5, 7	1, 3, 7
# of Channels	1, 6	3 - 256	3 - 512	3 - 1024	3 - 2048
# of Filters	6, 16	96 - 384	64 - 512	64 - 384	64 - 2048
Stride	1	1, 4	1	1, 2	1, 2
# of Weights	2.6k	2.3M	14.7M	6.0M	23.5M
# of MACs	283k	666M	15.3G	1.43G	3.86G
# of FC layers	2	3	3	1	1
# of Weights	58k	58.6M	124M	1M	2M
# of MACs	58k	58.6M	124M	1M	2M
Total Weights	60k	61M	138M	7M	25.5M
Total MACs	341k	724M	15.5G	1.43G	3.9G



Trend is Upwards, but not only...

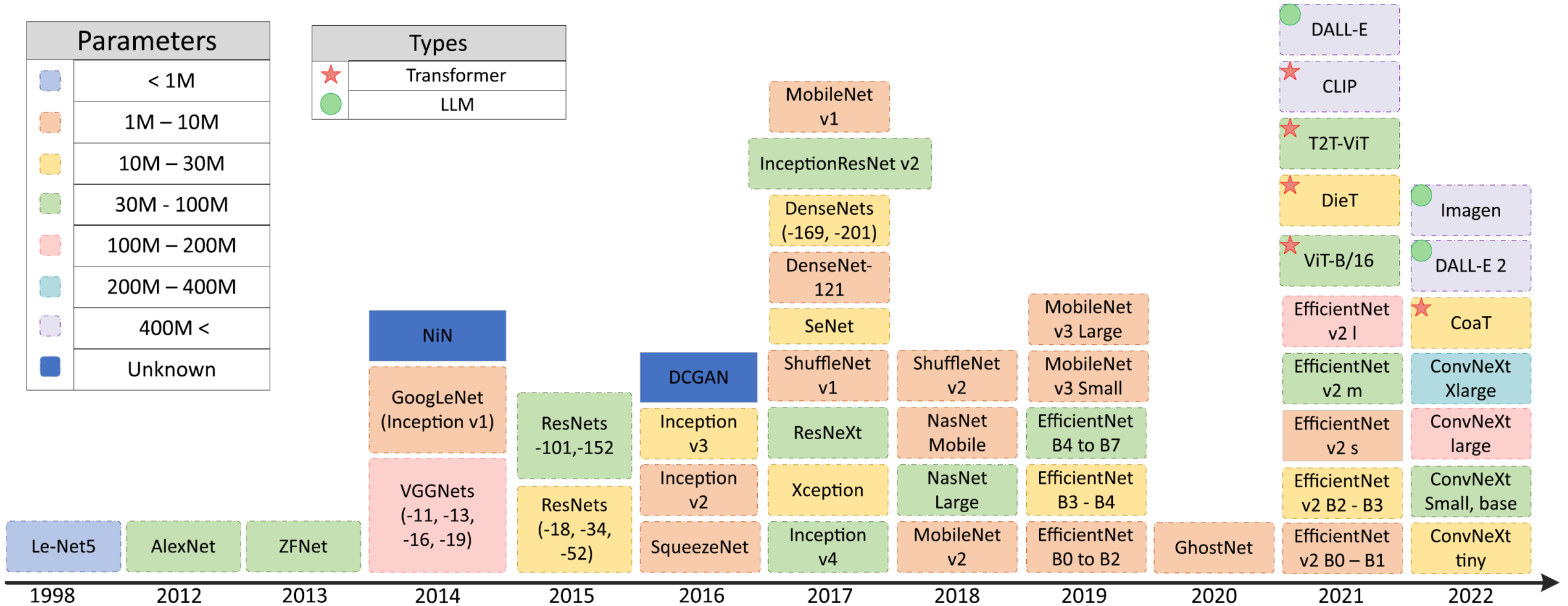


Image Classification Datasets

- **Image Classification/Recognition**
 - Given an entire image → Select 1 of N classes
 - No localization (detection)

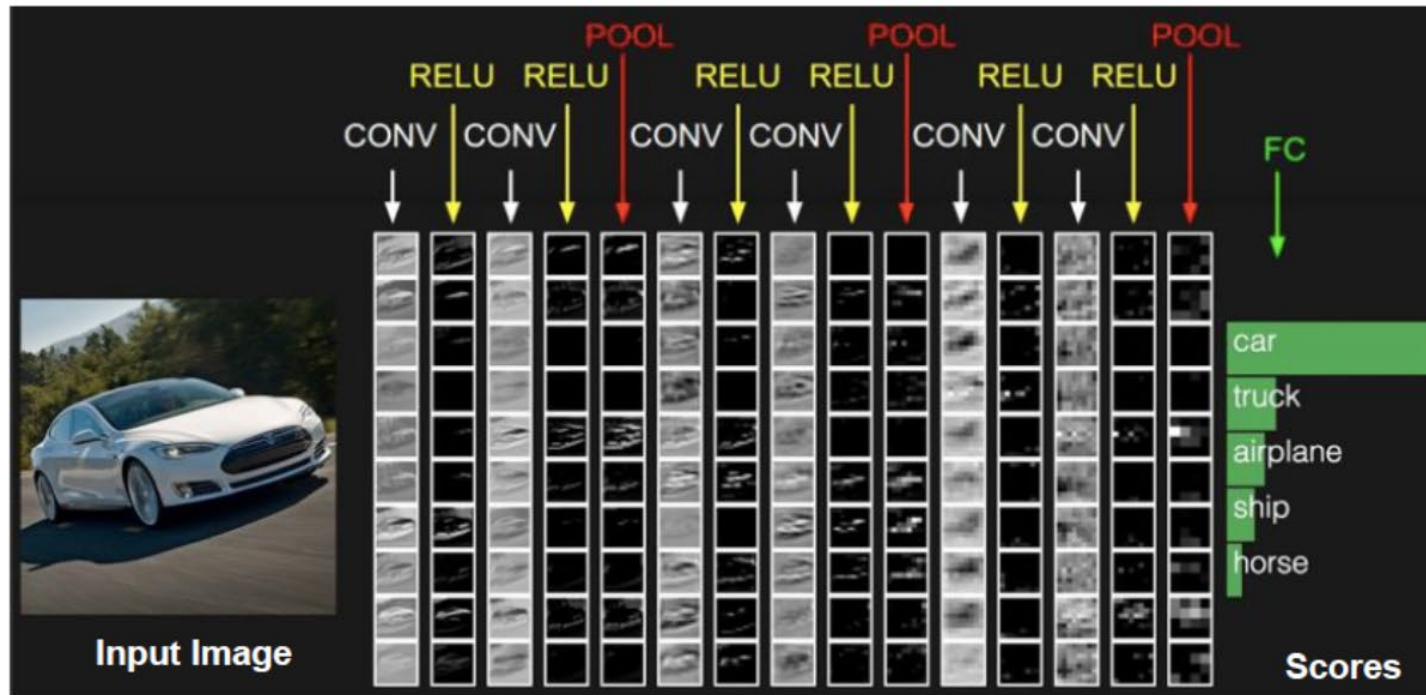


Image Source: Stanford cs231n

MNIST

Digit Classification

28x28 pixels (B&W)

10 Classes

60,000 Training

10,000 Testing

LeNet in 1998

(0.95% error)



ICML 2013

(0.21% error)



Object Classification

~256x256 pixels (color)

1000 Classes

1.3M Training

100,000 Testing (50,000 Validation)

Image Source: <http://karpathy.github.io/>

Image Source: <http://karpathy.github.io/>



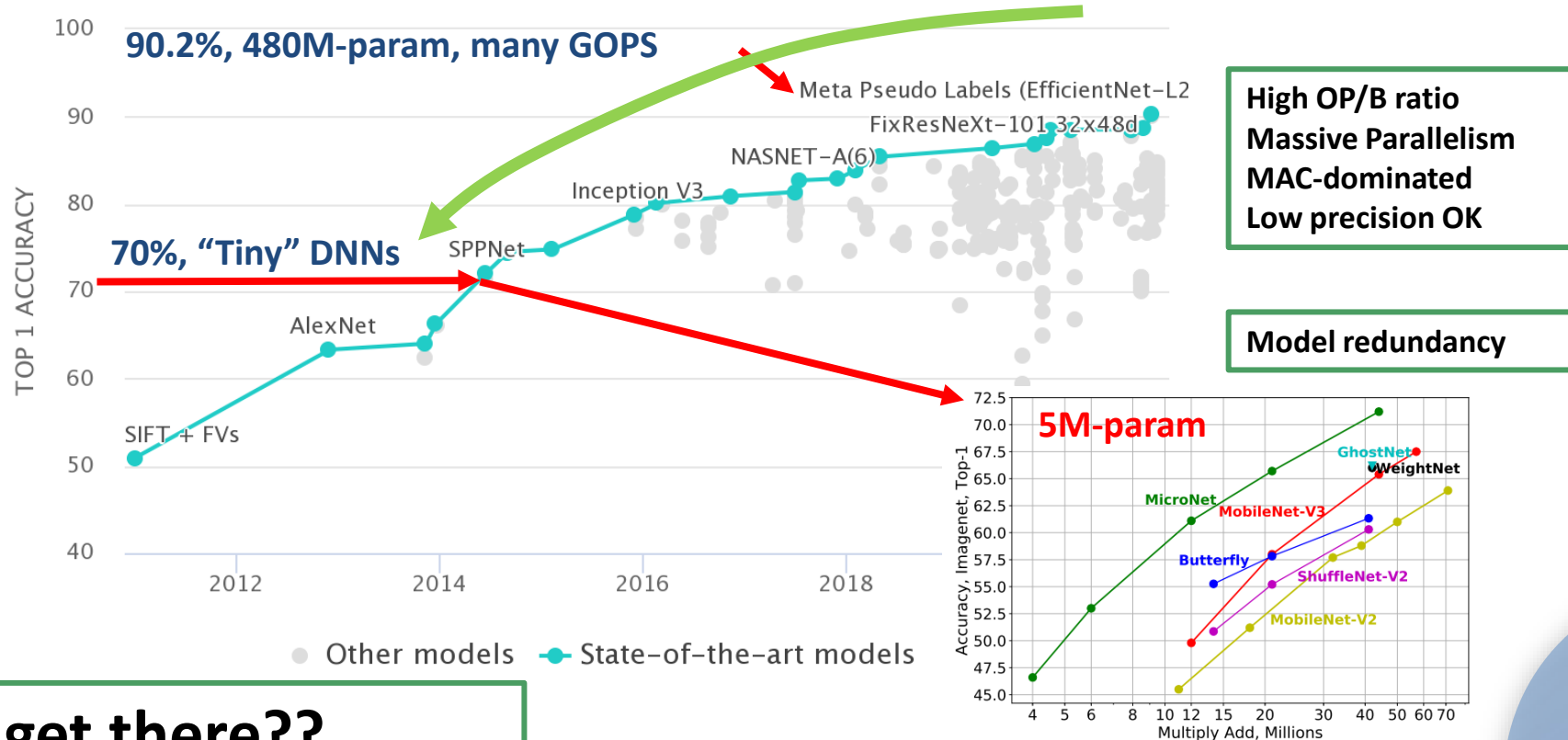
Image Classification Summary

	MNIST	IMAGENET
Year	1998	2012
Resolution	28x28	256x256
Classes	10	1000
Training	60k	1.3M
Testing	10k	100k
Accuracy	0.21% error (ICML 2013)	2.99% top-5 error (2016 winner)

[http://rodrigob.github.io/are_we_there_yet/build/
classification_datasets_results.html](http://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html)

From Huge to Small

H Pham 2021(Google) arXiv:2003.10580v3



How do we get there??

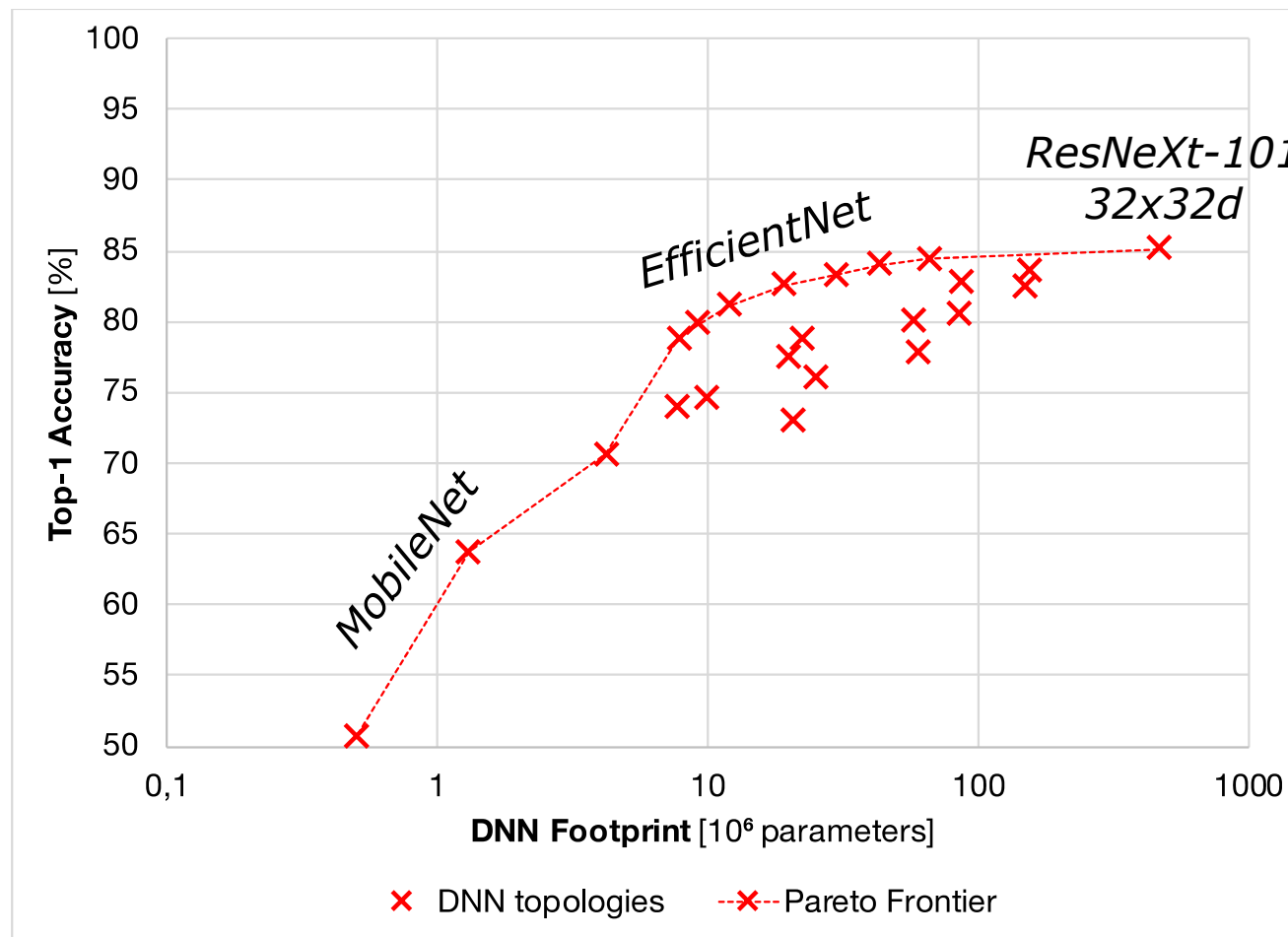
- Work at model level (separable convolutions, etc.)
- Work on reducing precision of parameters (quantization)
- Reduce model redundancy (pruning)



Small CNNs...

ImageNet Top-1 Accuracy vs **Memory Footprint**

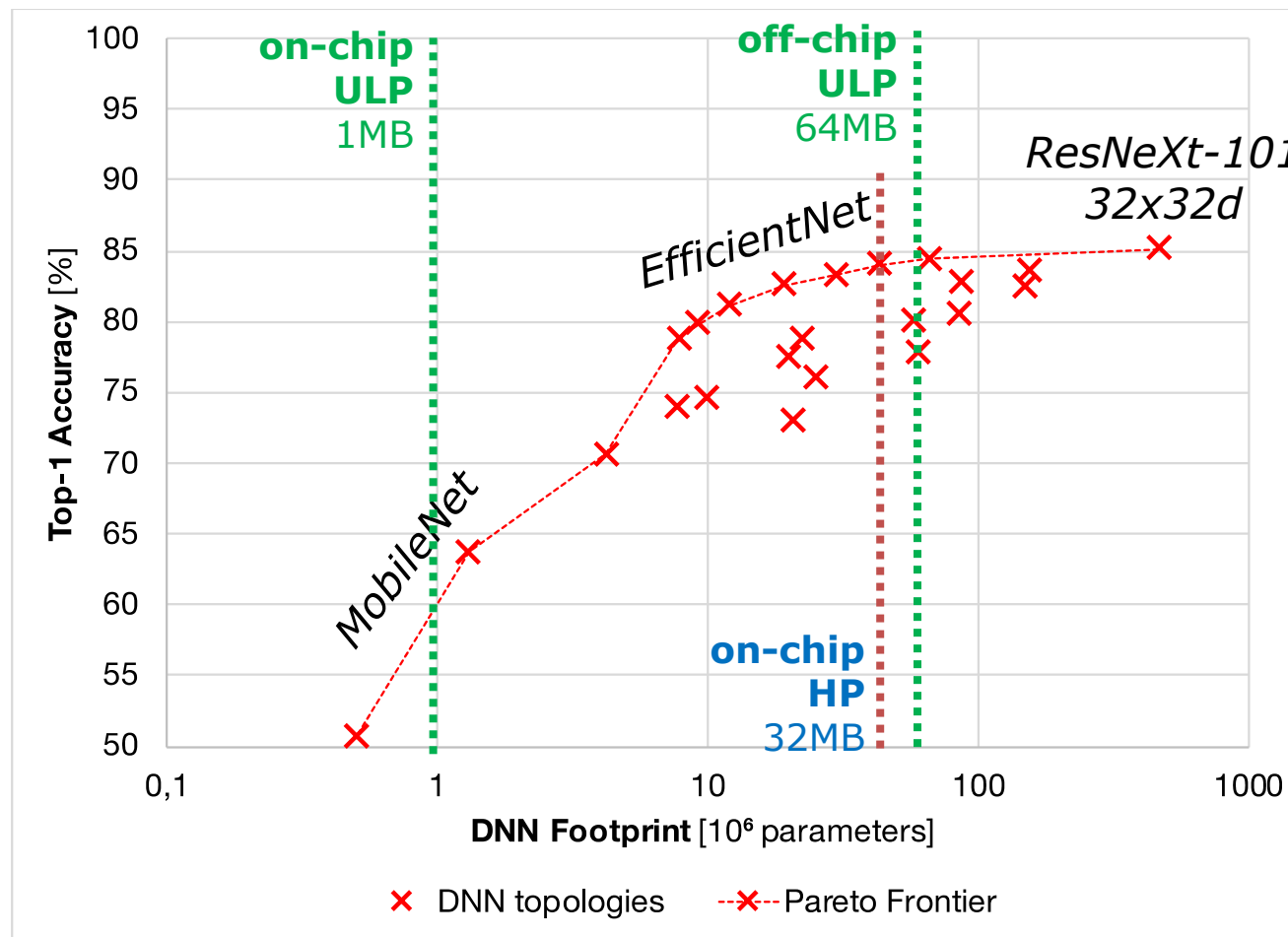
- Most entries > 10 MB
- Pareto Frontier Acc vs Memory (from 50% @ 0.5Mparam to 85% @ 445 Mparam)
- Not necessarily representative of real-world problems (might be more complex)



Small CNNs...

ImageNet Top-1 Accuracy vs Memory Footprint

- Most entries > 10 MB
- Pareto Frontier Acc vs Memory (from 50% @ 0.5Mparam to 85% @ 445 Mparam)
- Not necessarily representative of real-world problems (might be more complex)
- Almost always require off-chip DRAM even for **ULP**!



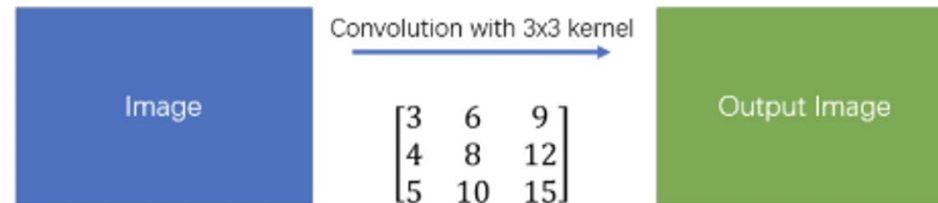
Spatially Separable Convolutions

At the level of single operators, complexity of CNNs can be reduced by means of separable convolutions.

- Separate the Conv computation into a combination of simpler operations.

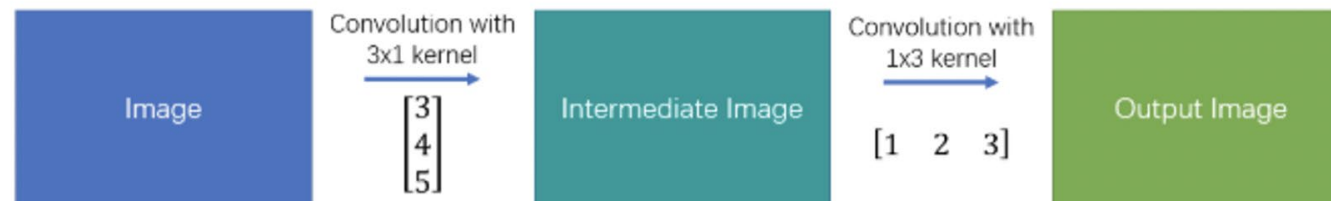
Spatially Separable Convolutions (earliest embodiment):

- Reduce a $K \times K$ 2D Conv to the combination of $K \times 1$ and $1 \times K$ 1D Convs
- E.g., for $K=3$, reduce the number of MULs from $3 \times 3 \times C_{in}$ to $2 \times (3 \times 1) \times C_{in}$



[Source] Wang, A Basic Introduction to Separable Convolutions.

Spatial Separable Convolution



Depthwise Separable Convolutions

Spatially Separable Conv. are not (yet?) very common in deep learning:

- Very few $K \times K$ kernels are spatially separable onto a combination of $K \times 1$ and $1 \times K$
- Training on the latter directly yields very low accuracy.

Depthwise Separable Conv is much more common

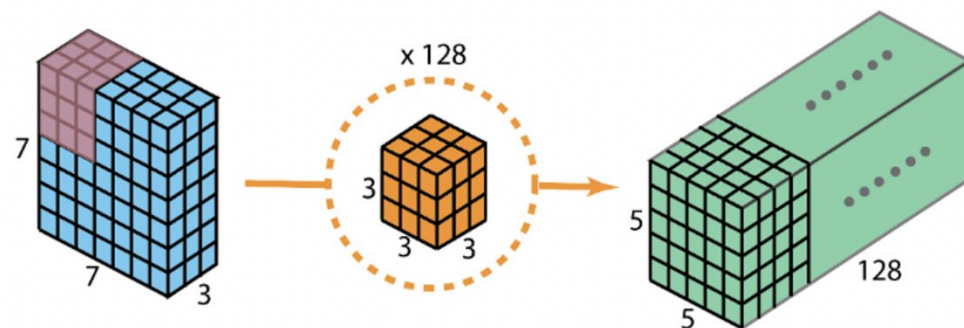
- Concept: separate over the depth (i.e., channels) dimension
- Use an additional **pointwise** (i.e., 1×1) Conv. to combine different channels
- First made popular by **MobileNets**

Howard et al, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

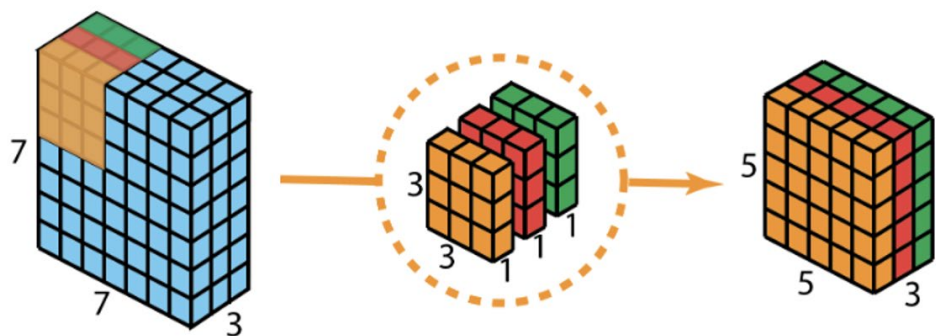


Depthwise Separable Convolutions

Standard Convolution

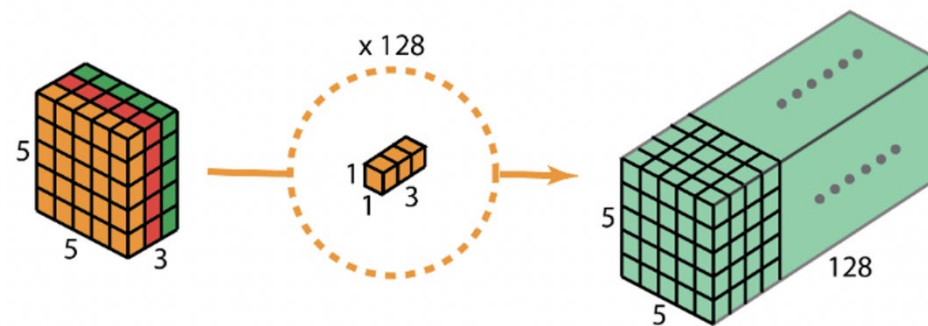


Depthwise Separable Convolution



+

Pointwise (1x1) Convolution



[Source] Bai, K. A Comprehensive Introduction to Different Types of Convolutions in Deep Learning.

Depthwise Separable Convolutions

Complexity:

- **Standard Conv:** $K * K * C_{in} * C_{out}$ operations (per output location) and weights
- **Depthwise (DW):** $K * K * C_{in}$ (Standard with $C_{out}=1$)
- **Pointwise (PW):** $1 * 1 * C_{in} * C_{out}$ (Standard with $K=1$)
- **(DW +PW) / Standard:** $1/C_{out} + 1/K^2$

Example: $K=3$, $C_{in} = 32$, $C_{out}=64$

- **Standard:** $3 * 3 * 32 * 64 = 18432$
- **DW + PW:** $3 * 3 * 32 + 1 * 1 * 32 * 64 = 288 + 2048 = 2336$ (**7.9x reduction!**)

DW + PW can be shown to approximate any general convolution:

- Guo et al, “Network Decoupling: From Regular to Depthwise Separable Convolutions”, 2018



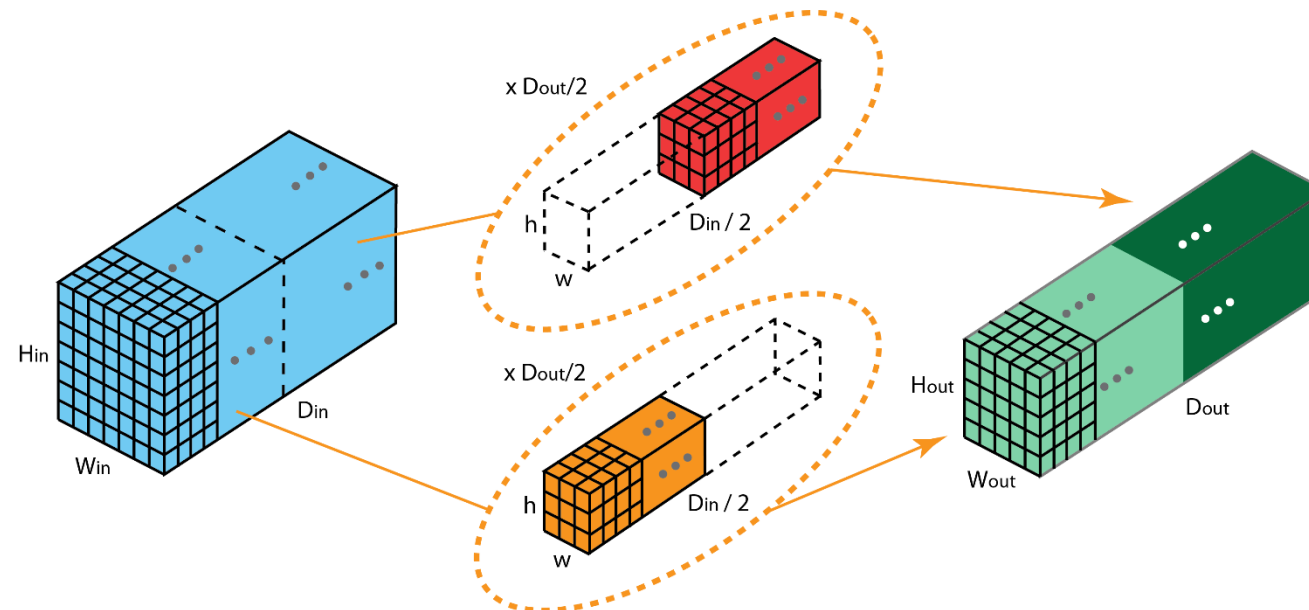
Grouped Convolutions

In Grouped, or GroupWise (GW) Convolutions, subsets of output channels are produced “looking at” subsets of input channels

Complexity: $G * (K * K * (C_{in}/G) * (C_{out}/G)) = 1/G * \text{Standard Conv}$

Example with $G = 2$ groups

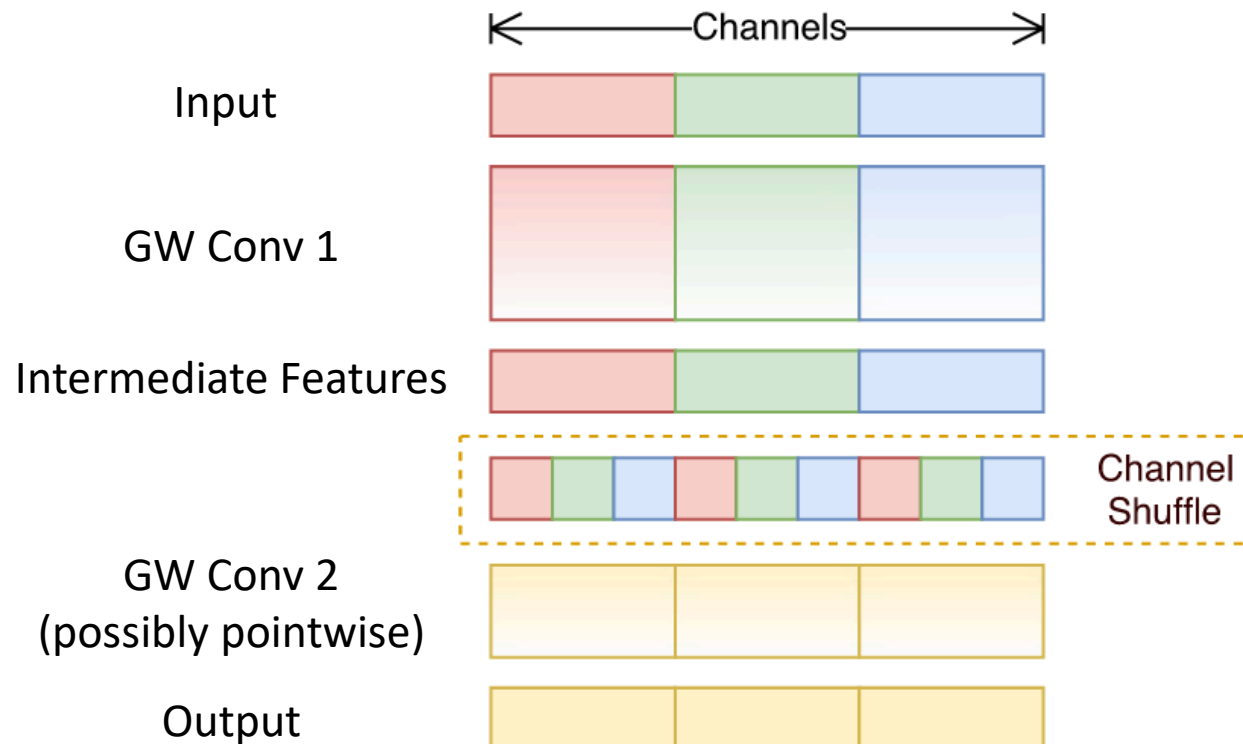
- DW Conv are a special case of Grouped Conv with $G = C_{in}$



Grouped Convolutions

Grouped Convolutions can be combined with channel reshuffling so that finally, each output takes information from all input channels:

- X. Zhang et al, *ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices*, 2017

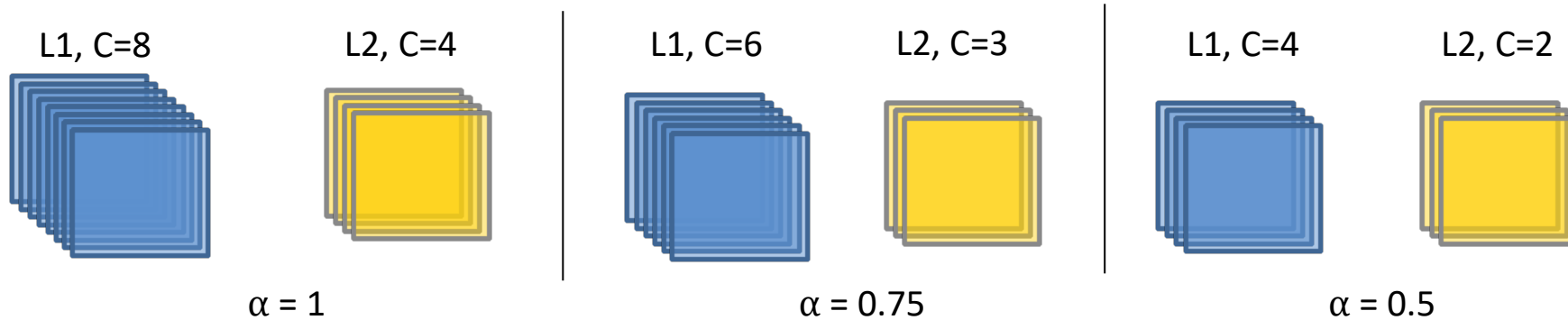


Width and Resolution Multipliers

Besides DW + PW Conv, MobileNetV1 also included two other ways to reduce complexity (at the expense of accuracy):

Width multiplier (α): uniform reduction of the # of channels in all layers

- Confusing name. Should be “depth multiplier”
- Reduce DW Conv by α and PW conv by α^2
- Reduce the n. weights of final FC layers

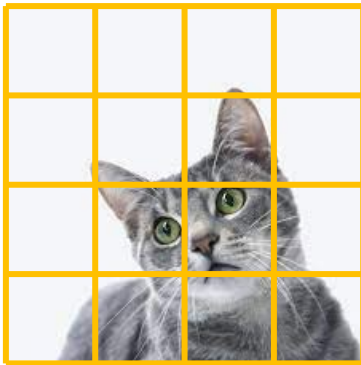


Width and Resolution Multipliers

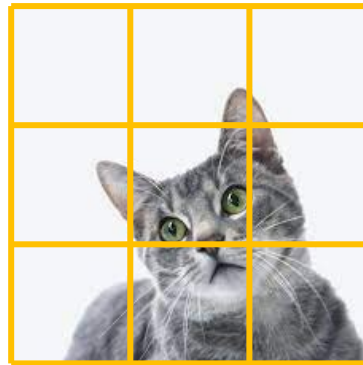
Besides DW + PW Conv, MobileNetV1 also included two other ways to reduce complexity (at the expense of accuracy):

Resolution multiplier (ρ): input resolution down-scaling

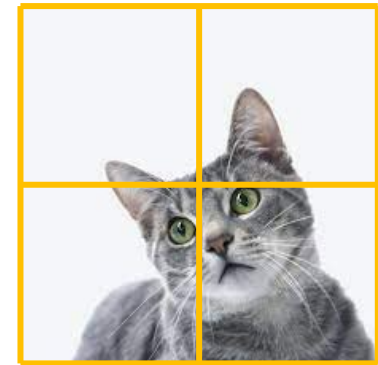
- Reduce the n. ops of all subsequent Conv layers by ρ^2
- Reduce the n. weights of final FC layers



$\rho = 1$



$\rho = 0.75$



$\rho = 0.5$

